

NATIONAL RESEARCH UNIVERSITY HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "HSE and University of London Double Degree Programme in Data
Science and Business Analytics"

Programming project report

on the topic Numerical Algorithms of Linear Algebra

Student:

group БПА/Д232

Sign

З. Ю. Зинкин

Name, Patronymic, Surname

4.02.2025

Date

Supervisor:

Дмитрий Витальевич Трушин

Name, Patronymic, Surname

доцент / ФКН НИУ ВШЭ

Position / Company

4.02.2025

Date

Grade

Sign

Moscow 2025

Contents

1	Introduction	3
1.1	Motivation	3
2	Description of functional and non-functional requirements for the program project	4
2.1	Functional requirements	4
2.2	Non-functional requirements	4
3	Theoretical part	5
3.1	Main definitions	5
3.2	Algorithms	7
3.2.1	Householder reflection	7
3.2.2	Givens rotations	8
3.2.3	QR decomposition	9
3.3	Schur Decomposition and QR Algorithm	11
3.3.1	Hessenberg Form and Hessenberg QR decomposition	11
3.3.2	Wilkinson Shifts	12
3.3.3	Real Schur Decomposition	12
4	Library LinAlgTools	13
4.1	Types	13
4.2	Core	14
4.3	Algorithms	14
5	Testing	15
5.1	Validation tests	15
5.2	Benchmark tests	15
5.2.1	Householder vs. Givens	16
6	Conclusion	18

Abstract

The main focus of the programming project is to study matrix decomposition algorithms from a theoretical point of view with its subsequent implementation on the C++ language.

1 Introduction

Matrix decomposition algorithms serve as fundamental tools in Numerical Linear Algebra and Computer Science, enabling efficient storage of large matrices, eigenvalue computation, solution of linear systems and least squares problems, among other applications. This broad utility necessitates not only awareness of their practical uses, but also a deep understanding of their theoretical foundations and implementation details.

This programming project aims to study modern Numerical Linear Algebra algorithms, with particular emphasis on matrix decompositions, which represent a matrix in a certain way, as a product of other matrices, for further computation. In order to have a profound understanding of each algorithm, it is necessary to implement a C++ library from scratch.

C++ is the language of choice for the project, since it provides high performance through low-level memory control and compiler optimization. Moreover, its specialization in object-oriented design and template system make it suitable for working with matrices.

The core of the implementation centers around a matrix class that handles matrix representation, arithmetic operations, and supporting methods. All decomposition algorithms build upon this fundamental class. The entire library is tested using GoogleTest[4].

The theoretical part (3) provides mathematical background for each algorithm. Implementation details are thoroughly documented in the Library `LinAlgTools` (4) section. Test results and validation appear in the Testing (5) section.

The information for the theoretical part of the project is taken from the books, which are cited at the end of the paper. Citation of exact chapters and theorems is used where necessary.

The result of the project is a C++ library `LinAlgTools`[11] with the following functionality:

- Matrix arithmetic
- QR Decomposition
- Hessenberg Form
- Schur decomposition
- Bidiagonalization
- Singular Value Decomposition (SVD)

1.1 Motivation

Before proceeding with theoretical and technical details, it is essential to highlight the importance of this work and compare it to existing solutions. While the motivation for learning decomposition algorithms has already been outlined, the incentive behind the technical implementation must also be addressed. Let us first discuss existing libraries with similar functionality.

- *Eigen*[5] - a high-performance library, supporting dense and sparse matrices, solvers and many decomposition algorithms. However, due to prioritization of performance over readability, the code becomes unreadable. Moreover, the syntax of the library can be overwhelming.
- *Armadillo*[8] - a library, aiming towards a good balance between speed and ease of use. Decomposition algorithms are provided through integration with LAPACK[2], which relies on highly optimized, but opaque implementation. Therefore, the library's implementation may not be clear for an inexperienced user.
- *Dlib*[7] - a machine learning library with readable C++11 code and thorough documentation. However, its age makes it less appealing than Eigen (which updates regularly) for professional use. Due to older C++ standard the library does not utilize powerful additions to the language, such as concepts, to drastically reduce the size of code. More importantly, the source code is flooded with inexpressive variable names, which make it difficult to follow.

As demonstrated, neither of the libraries are capable of providing clear and comprehensive implementation of the algorithms we need. They either prioritize performance at the expense of clarity or suffer from obsolete design choices. Therefore, `LinAlgTools` is developed as a learning tool, which can be used to properly understand each algorithms' implementation by any C++ amateur.

2 Description of functional and non-functional requirements for the program project

2.1 Functional requirements

- A *Matrix* class, which allows users to perform matrix arithmetic. The class must be able to support both real and complex elements. Moreover, the class must have a clear and intuitive interface.
- A *SubMatrix* and *ConstSubMatrix* classes for handling a particular part of an existing matrix without copying data. These classes must fully replicate the interface of the base *Matrix* class to ensure complete compatibility. Furthermore, cross-type operations must be implemented to perform arithmetic between objects of *Matrix* and *SubMatrix* classes of appropriate sizes.
- Additional utility functions are to be added. They must perform the following functionality: calculate the sign of a real or complex number, distinguish between real and complex types, verify whether a given number is close 0, check if two matrices are equal with a possible error of a pre-determined ε , and more.
- Implementation of numerically stable algorithms:
 - QR Decomposition - representation of a matrix as a product of Q and R matrices, where Q is unitary and R is upper-triangular. The decomposition must be implemented using both Householder reflections and Givens rotations.
 - Hessenberg Form - representation of a matrix as a product of U , H and U^* matrices, where U is unitary and H is upper Hessenberg matrix.
 - Schur Decomposition - representation of a matrix as a product of U , T and U^* , where U is unitary and T is upper-triangular.
 - Bidiagonalization - representation of a matrix as a product of U , B and V^* matrices, where U , V are unitary and B is bidiagonal.
 - SVD Decomposition - representation of a matrix as a product of U , Σ and V^* matrices, where U , V are unitary and Σ is a diagonal matrix.
- Comprehensive testing is essential to ensure the library's reliability. This includes validation on both square and rectangular matrices with real and complex entries, along with tests on potential edge cases.
- Randomized benchmark testing should be performed, generating multiple matrices of specified sizes per iteration. Each test should measure decomposition execution times, recording the mean, minimum, and maximum durations in a dedicated file for subsequent performance analysis.

2.2 Non-functional requirements

- Implementation of the library on the C++ language without third-party libraries, except for GoogleTest.
- *Git*, a version control system, and a remote repository on *GitHub* [11].
- *Clang* compiler with C++23 standard support.
- Code formatter *clang-format* to automatically format code based on the Microsoft style with additional changes.
- Static code analyzer *clang-tidy* to identify potential errors and to stick to uniform naming of variables, functions, etc.
- A third-party library *Googletest* [4] for writing tests.
- A text editor with IDE capabilities *Neovim*.
- Minimum RAM requirement: 4GB.

3 Theoretical part

Theoretical part is divided into two parts: *Definitions* and *Algorithms*. The first part establishes rigorous framework for the algorithm discussed later. The second part discusses implemented algorithms from a theoretical point of view by providing verbal description, mathematical statements with proofs and pseudocode. Special emphasis is made on proofs, as they not only mathematically validate the algorithms, but often reveal their structure. Pseudocode sections deliberately omit edge-cases to focus on core ideas and maintain clarity.

The paper assumes familiarity with Linear Algebra concepts, allowing for the omission of elementary explanations.

3.1 Main definitions

Remark 1. A field $\mathbb{F}$ will denote either $\mathbb{R}$ or $\mathbb{C}$.

Remark 2. A set of all $m \times n$ matrices over a field $\mathbb{F}$ we denote $\mathbb{F}^{m \times n}$.

Remark 3. An $n \times n$ Identity matrix will be denoted I_n .

Remark 4. A set of all $n \times 1$ or $1 \times n$ vectors over $\mathbb{F}$ we denote $\mathbb{F}^n$.

Remark 5. The i -th standard basis vector of a vector space $\mathbb{F}^m$ we denote $e_i = [0, \dots, 1, \dots, 0]$

Definition 1. The subdiagonal of a matrix is the set of elements directly below the elements comprising the diagonal.

Definition 2. A matrix $A \in \mathbb{F}^{m \times n}$ is called *upper Hessenberg* if all its elements below the subdiagonal are 0:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1k} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2k} & \dots & a_{2n} \\ 0 & a_{32} & a_{33} & \dots & a_{3k} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{mk} & \dots & a_{mn} \end{bmatrix}$$

Definition 3. The superdiagonal of a matrix is the set of elements directly above the elements comprising the diagonal.

Definition 4. A matrix $A \in \mathbb{F}^{m \times n}$ is called *bidiagonal* if all its elements below the diagonal and above the superdiagonal are 0:

$$\begin{bmatrix} a_{11} & a_{12} & 0 & \dots & 0 & 0 & \dots & 0 \\ 0 & a_{22} & a_{23} & \dots & 0 & 0 & \dots & 0 \\ 0 & 0 & a_{33} & \dots & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{mk} & a_{mk+1} & \dots & 0 \end{bmatrix}$$

Definition 5. A *Hermitian conjugate* of a matrix $A \in \mathbb{C}^{m \times n}$ is an $n \times m$ matrix, obtained by transposing A and applying complex conjugate to each of entry: $(A^*)_{ij} = \overline{A_{ji}}$.

Remark. For real matrices *Hermitian conjugate* coincides with transposition of the matrix: $A^* = A^T$.

Definition 6. A matrix $A \in \mathbb{C}^{m \times n}$ is called *Normal* if $A^*A = AA^*$.

Definition 7. A matrix $A \in \mathbb{C}^{m \times n}$ is called *Hermitian* if $A^* = A$.

Definition 8. A matrix $A \in \mathbb{R}^{m \times n}$ is called *Orthogonal* if $A^T = A^{-1}$. As a consequence, $A^T A = AA^T = I$.

Definition 9. A matrix $A \in \mathbb{C}^{m \times n}$ is called *Unitary* if $A^* = A^{-1}$. As a consequence, $A^*A = AA^* = I$.

Statement 1. A product of unitary (orthogonal) matrices is also unitary (orthogonal).

Proof. Let U and V be unitary:

$$(UV)^*UV = V^*U^*UV = V^*V = I$$

□

Definition 10. Matrices $A \in \mathbb{C}^{n \times n}$ and $B \in \mathbb{C}^{n \times n}$ are called *similar* if exists a matrix $C \in \mathbb{C}^{n \times n}$, such that $A = C^{-1}BC$.

Definition 11. A *scalar product* is a bilinear form $\langle \cdot, \cdot \rangle : \mathbb{F}^n \times \mathbb{F}^m \rightarrow \mathbb{F}$. In a Euclidean space, such as $\mathbb{R}^m$ or $\mathbb{C}^m$ the scalar product of two vector $v, w \in \mathbb{C}^m$ is defined in the following way:

$$\langle v, w \rangle = w^* v = \sum_{i=1}^m v_i \overline{w_i}$$

Definition 12. A 2-norm, or Euclidean norm, on an Euclidean space is a function: $\|\cdot\|_2 : \mathbb{F}^n \rightarrow \mathbb{R}_+$.
Let $v \in \mathbb{F}^n$:

$$\|v\|_2 = \sqrt{\langle v, v \rangle} = \sqrt{\sum_{i=1}^m |v_i|^2}$$

Remark. Geometrically, a 2-norm returns the distance from the origin to the point in the space.

Definition 13. A matrix is called *sparse* if the majority of its elements are 0.

Definition 14. A matrix is called *dense* if the majority of its elements are non-zero.

Remark. There is no strict threshold to classify a matrix as sparse or dense, but a common convention considers a matrix sparse if the number of non-zero elements is approximately equal to the number of rows or columns.

3.2 Algorithms

3.2.1 Householder reflection

Definition 15. A *Householder reflection* is a linear transformation which reflects a vector about a hyperplane[3].

Given a vector $v \in \mathbb{F}^m : \|v\| = 1$, an $m \times m$ matrix P (or more commonly $H(v)$) of the form

$$P = H(v) = I - 2vv^*$$

is called a *Householder reflector* or *Householder matrix*. The matrix allows us to reflect vector along the hyperplane, spanned by v . Moreover, a Householder matrix is unitary and hermitian ($H^{-1} = H^* = H$), which turns out to be key in the QR decomposition algorithm.

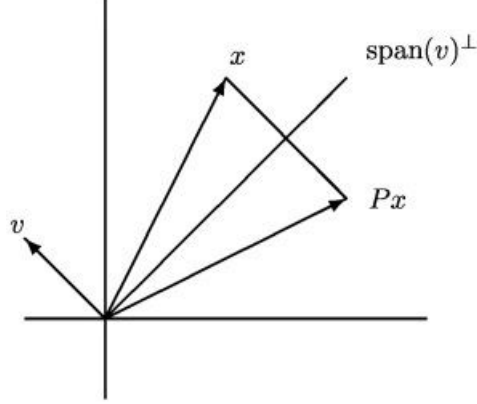


Figure 1: Householder reflection applied to a vector x

Statement 2. $\forall a, b \in \mathbb{C}^n : \|a\|_2 = \|b\|_2, \exists \gamma \in \mathbb{C} : |\gamma| = 1$ and $\exists v \in \mathbb{C}^n : \|v\|_2 = 1$, such that $H(v)a = \gamma b$.

Proof. By direct computation we can find the required γ and v :

$$H(v) = I - 2vv^*$$

$$H(v)a = a - 2vv^*a = \gamma b$$

$$2(v^*a)v = a - \gamma b \tag{1}$$

From here it follows that v must be of the form:

$$v = \mu \frac{a - \gamma b}{\|a - \gamma b\|_2}$$

Since we are proving the existence, we can set μ to 1. Therefore

$$v = \frac{a - \gamma b}{\|a - \gamma b\|_2}$$

Plugging it into (1):

$$\begin{aligned} 2 \frac{(a - \gamma b)^*}{\|a - \gamma b\|_2} a \frac{a - \gamma b}{\|a - \gamma b\|_2} &= a - \gamma b \\ 2(a - \gamma b)^* a &= \|a - \gamma b\|_2^2 \\ 2(a - \gamma b)^* a &= (a - \gamma b)^* (a - \gamma b) \\ 2a^* a - 2\bar{\gamma}b^* a &= a^* a + b^* b - \gamma a^* b - \bar{\gamma}b^* a \\ \bar{\gamma}b^* a &= \text{Re}(\bar{\gamma}b^* a) \\ \gamma &= \pm \frac{b^* a}{|b^* a|} \end{aligned}$$

If $a \perp b$, γ can be any number, such that $|\gamma| = 1$. If $a \parallel b$, then we should carefully choose the sign to guarantee numeric stability of the transformation. $\square$

Corollary 1. $\forall x \in \mathbb{C}^n \exists v \in \mathbb{C}^n : \|v\|_2 = 1$ and

$$H(v)x = \gamma \begin{bmatrix} \|x\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$v = \frac{x - \gamma \|x\|_2 e_1}{\|x - \gamma \|x\|_2 e_1\|_2}, \gamma = -\frac{x_1}{|x_1|}$$

The proof and corollary yield an algorithm for vector transformation. To apply this transformation to a matrix from the left (or right), we use:

Algorithm 1 Householder Left Reflection

```

1: function HOUSEHOLDERLEFTREFLECTION( $A \in \mathbb{F}^{m \times n}$ ,  $x \in \mathbb{F}^m$ )
2:    $v = x$ 
3:    $v_1 = x_1 - \text{sign}(x_1) \cdot \|x\|_2$ 
4:    $v = v \cdot \frac{1}{\|v\|}$ 
5:    $M = A - (2v) \cdot (v^* A)$ 
6:   return M
7: end function

```

3.2.2 Givens rotations

Definition 16. A *Givens rotation* is a linear transformation which amounts to a counterclockwise rotation by an angle of θ [3].

In case we need to zero specific elements of the matrix, *Givens rotations* are our tool of choice. The matrix of a *Givens rotation* is of the form ($c = \cos \theta$, $s = \sin \theta$):

$$G(i, j, \theta) = \begin{bmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \cdots & c & \cdots & -s & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & s & \cdots & c & \cdots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{bmatrix} \begin{matrix} \\ \\ i \\ \\ j \\ \\ \end{matrix}$$

Statement 3. $\forall v \in \mathbb{F}^m, i, j \in \mathbb{N}_+ \exists \theta \in \mathbb{F} :$

$$G(i, j, \theta) \begin{bmatrix} \vdots \\ a \\ \vdots \\ b \\ \vdots \end{bmatrix} = \begin{bmatrix} \vdots \\ r \\ \vdots \\ 0 \\ \vdots \end{bmatrix} \begin{matrix} i \\ j \end{matrix}$$

Proof. If $x \in \mathbb{F}^m$ and

$$y = G(i, j, \theta)x$$

Then:

$$y_j = \begin{cases} cx_i - sx_k, & j = i \\ sx_i + cx_k, & j = k \\ x_j, & j \neq i, k \end{cases}$$

From here we can deduct that $y_k = 0$ if:

$$c = \frac{|x_i|}{\sqrt{|x_i|^2 + |x_k|^2}}, \quad s = \frac{-\bar{x}_k}{\sqrt{|x_i|^2 + |x_k|^2}} \cdot \frac{x_i}{|x_i|}$$

□

Notice that G is almost the identity matrix. We can exploit this structure to achieve faster computation by apply the transformation directly to the rows which are affected by G .

Algorithm 2 Givens Left Rotation

```

1: function GIVENSLEFTROTATION( $A \in \mathbb{F}^{m \times n}$ ,  $i \in \mathbb{N}_+$ ,  $j \in \mathbb{N}_+$ ,  $a \in \mathbb{F}$ ,  $b \in \mathbb{F}$ )
2:    $r = \sqrt{|a|^2 + |b|^2}$ 
3:    $c = a/r$ 
4:    $s = -b/r$ 
5:    $M = A$ 
6:   for  $k = 1$  to  $n$  do
7:      $M_{ik} = \bar{c} \cdot A_{ik} - \bar{s} \cdot A_{jk}$ 
8:      $M_{jk} = s \cdot A_{ik} + c \cdot A_{jk}$ 
9:   end for
10:  return  $M$ 
11: end function

```

3.2.3 QR decomposition

The *QR Decomposition* serves as a fundamental tool for solving linear least squares problems and computing matrix eigenvalues. Its primary motivation in this implementation stems from enabling the *QR algorithm* (discussed later). The decomposition's numerical stability and utility across algorithms make it indispensable for our purposes.

Theorem 1. For any matrix $A \in \mathbb{F}^{m \times n}$ exist a unitary matrix $Q \in \mathbb{F}^{m \times m}$ and an upper-triangular matrix $R \in \mathbb{F}^{m \times n}$, such that $A = QR$.

Proof. Using Householder rotations (for convenience $A \in \mathbb{F}^{4 \times 3}$):

$$\begin{aligned}
A &= \begin{bmatrix} \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \end{bmatrix} \xrightarrow{H(v_1)} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \end{bmatrix} \xrightarrow{H(v_2)} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \\ 0 & 0 & \star \end{bmatrix} \xrightarrow{H(v_3)} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \\ 0 & 0 & 0 \end{bmatrix} = R \\
&\quad H_3 H_2 H_1 A = R \\
A &= H_1^{-1} H_2^{-1} H_3^{-1} R = H_1 H_2 H_3 R = QR
\end{aligned}$$

□

Algorithm 3 Householder QR Decomposition

```

1: function HOUSEHOLDERQR( $A \in \mathbb{F}^{m \times n}$ )
   Complexity:  $\mathcal{O}(4mn^2 - \frac{4}{3}n^3)$ 
2:    $Q = I_m$ 
3:    $R = A$ 
4:   for  $k = 1$  to  $\min(m, n)$  do
5:      $v = R[k : m, k]$ 
6:      $R[k : m, k : n] = \text{HouseholderLeftReflection}(R[k : m, k : n], v)$ 
7:      $Q[k : m, 1 : m] = \text{HouseholderLeftReflection}(Q[k : m, 1 : m], v)$ 
8:   end for
9:    $Q = Q^*$ 
10:  return  $Q, R$ 
11: end function

```

Proof. Using Givens rotations (for convenience $A \in \mathbb{F}^{3 \times 2}$):

$$A = \begin{bmatrix} \star & \star \\ \star & \star \\ \star & \star \end{bmatrix} \xrightarrow{G_{13}} \begin{bmatrix} \star & \star \\ \star & \star \\ 0 & \star \end{bmatrix} \xrightarrow{G_{12}} \begin{bmatrix} \star & \star \\ 0 & \star \\ 0 & \star \end{bmatrix} \xrightarrow{G_{23}} \begin{bmatrix} \star & \star \\ 0 & \star \\ 0 & 0 \end{bmatrix} = R$$

$$G_{23}G_{12}G_{13}A = R$$

$$A = G_{13}^*G_{12}^*G_{23}^*R = QR$$

□

Algorithm 4 Givens QR Decomposition

```

1: function GIVENSQR( $A \in \mathbb{F}^{m \times n}$ )
   Complexity:  $\mathcal{O}(6mn^2 - \frac{5}{2}n^3)$ 
2:    $Q = I_m$ 
3:    $R = A$ 
4:   for  $k = 1$  to  $\min(m, n)$  do
5:     for  $i = m$  downto  $k + 1$  do
6:        $a = R[i - 1, k]$ 
7:        $b = R[i, k]$ 
8:       if  $b \neq 0$  then
9:          $R[1 : m, k : n] = \text{GivensLeftRotation}(R[1 : m, k : n], i - 1, i, a, b)$ 
10:         $Q[1 : m, 1 : m] = \text{GivensRightRotation}(Q[1 : m, 1 : m], i - 1, i, a, b)$ 
11:       end if
12:     end for
13:   end for
14:   return  $Q, R$ 
15: end function

```

While Householder reflections appear superior to Givens rotations due to better computational complexity, this advantage is not universal. Givens rotations excel at zeroing specific elements, making them particularly effective for sparse matrices or matrices nearly upper-triangular (see [1] for sparse orthogonal decomposition). Conversely, Householder reflections demonstrate better performance on dense matrices through efficient full-column operations.

Consequently, each approach presents distinct advantages and limitations that must be carefully considered when working with structured matrices.

3.3 Schur Decomposition and QR Algorithm

We now turn to the *Eigenvalue Problem* - the computation of a matrix's eigenvalues and corresponding eigenvectors.

Theorem 2. (Schur) $\forall A \in \mathbb{C}^{n \times n}$ exist unitary $U \in \mathbb{C}^{n \times n}$ and an upper-triangular $T \in \mathbb{C}^{n \times n}$, such that $A = UTU^*$.

Proof. Theorem 7.1.3 [3]. □

However, the proof does not provide a constructive algorithm, necessitating the introduction of a computational method to enable this decomposition and subsequent SVD. The eigenvalue problem is either computationally inefficient or numerically unstable. These problems suggest to shift our focus to iterative algorithms and examine their convergence properties. The *QR Algorithm*, when applied to a square matrix A , converges to an upper-triangular matrix T from the Schur decomposition.

Algorithm 5 QR Algorithm

```

1: function ALGORITHMQR( $A \in \mathbb{F}^{n \times n}$ )
   Complexity:  $\mathcal{O}(n^4)$ 
2:    $A_0 = A$ 
3:   for  $k = 1, 2 \dots$  do
4:      $A_{k-1} = Q_k R_k$ 
5:      $A_k = R_k Q_k$ 
6:   end for
7:   return  $A_k$ 
8: end function

```

Proof. The general convergence proof is quite involved (see [10]), while a simpler version for normal matrices appears in [9]. □

While the above pseudocode represents the algorithm's simplest form, its convergence is often impractically slow or may fail entirely. Two particularly effective enhancements are *matrix preprocessing* and introduction of *shift*. The latter ensures universal convergence with at worst quadratic complexity.

3.3.1 Hessenberg Form and Hessenberg QR decomposition

As previously mentioned, matrix preprocessing can dramatically improve the algorithm's computational efficiency. In modern practice, transforming the matrix to *Hessenberg form* precedes application of the QR algorithm.

Theorem 3. $\forall A \in \mathbb{C}^{n \times n}$ exist unitary $U \in \mathbb{C}^{n \times n}$ and an upper Hessenberg $H \in \mathbb{C}^{n \times n}$, such that $A = UHU^*$.

Proof. The proof is identical to the that of the Householder QR decomposition with an only change of having to zero out the subdiagonal elements. □

Algorithm 6 Hessenberg form

```

1: function HESSENBERGFORM( $A \in \mathbb{F}^{n \times n}$ )
   Complexity:  $\mathcal{O}(\frac{10}{3}n^3)$ 
2:    $U = I_n$ 
3:    $H = A$ 
4:   for  $k = 1$  to  $n$  do
5:      $v = H[k+1 : n, k]$ 
6:      $H[k+1 : n, k : n] = \text{HouseholderLeftReflection}(H[k+1 : n, k : n], v)$ 
7:      $H[1 : n, k+1 : n] = \text{HouseholderRightReflection}(H[1 : n, k+1 : n], v)$ 
8:      $U[k+1 : n, 1 : n] = \text{HouseholderLeftReflection}(U[k+1 : n, 1 : n], v)$ 
9:   end for
10:   $U = U^*$ 
11: end function

```

The similarity between A and H guarantees identical eigenvalues, permitting the QR algorithm to operate on the Hessenberg structure with improved stability and convergence. Additionally, the Hessenberg form reduces the QR decomposition complexity to $\mathcal{O}(n^2)$ by limiting elimination to subdiagonal elements.

Algorithm 7 Hessenberg QR Decomposition

```

1: function HESSENBERGQR( $A \in \mathbb{F}^{m \times n}$ )
   Complexity:  $\mathcal{O}(n^2)$ 
2:    $Q = I_m, R = A$ 
3:   for  $k = 1$  to  $\min(m, n)$  do
4:      $a = R[i, j]$ 
5:      $b = R[i + 1, j]$ 
6:      $R[1 : m, k : n] = \text{GivensLeftRotation}(R[1 : m, k : n], i, i + 1, a, b)$ 
7:      $Q[1 : m, 1 : m] = \text{GivensRightRotation}(Q[1 : m, 1 : m], i, i + 1, a, b)$ 
8:   end for
9:   return  $Q, R$ 
10: end function

```

3.3.2 Wilkinson Shifts

An alternative approach for enhancing convergence, and in some cases enabling convergence to exact solutions, is the use of *Wilkinson Shifts*. Since their effectiveness is documented in the QR algorithm's convergence proof [10], we omit detailed discussion here.

Definition 17. Given a matrix $A \in \mathbb{F}^{n \times n}$:

$$A = \begin{bmatrix} \ddots & \vdots & \vdots \\ \dots & a & b \\ \dots & c & d \end{bmatrix} \quad \delta = \frac{a + d}{2}$$

A Wilkinson shift is a number:

$$\mu = d + \delta - \text{sign}(\delta) \sqrt{\delta^2 + b + c}$$

3.3.3 Real Schur Decomposition

Theorem 4. $\forall A \in \mathbb{R}^{n \times n}$ exists an orthogonal $Q \in \mathbb{R}^{n \times n}$, such that:

$$Q^T A Q = \begin{bmatrix} R_{11} & R_{12} & \dots & R_{1m} \\ 0 & R_{21} & \dots & R_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & R_{mm} \end{bmatrix}$$

where each R_{ii} is either a 1×1 or a 2×2 matrix having complex conjugate eigenvalues.

Proof. Theorem 7.4.1 [3] □

Algorithm 8 Real Schur Decomposition

```

1: function REALSCHUR( $A \in \mathbb{R}^{n \times n}$ )
   Complexity:  $\mathcal{O}(n^3)$ 
2:    $U_0, S_0 = \text{HessenbergForm}(A)$ 
3:   for  $k = 1, 2 \dots$  do
4:      $\alpha = \text{GetWilkinsonShift}(S_{k-1})$ 
5:      $Q_k, R_k = \text{HouseholderQR}(S_{k-1} - \alpha I_n)$  ▷ Utilizes HessenbergQR if structure is detected
6:      $S_k = R_k Q_k + \alpha * I_n$ 
7:      $U_k = U_k Q_k$ 
8:   end for
9:   return  $U_k, S_k$ 
10: end function

```

We may therefore regard the Real Schur Decomposition as an *Eigenvalue-Revealing Decomposition*, since the real and imaginary parts of all eigenvalues can be extracted directly from the diagonal blocks R_{ii} .

4 Library LinAlgTools

The library is designed with a modular and extensible architecture to efficiently handle decomposition algorithms. It has the following architecture:

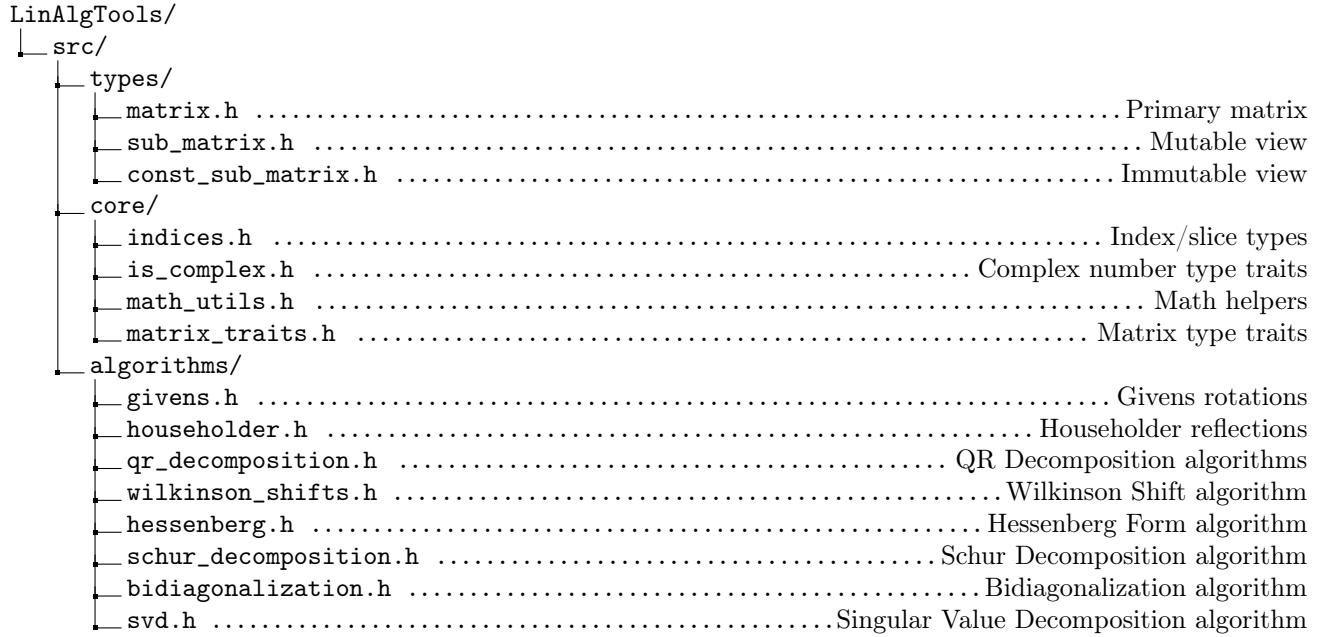


Figure 2: Directory structure of the LinAlgTools library

4.1 Types

This part of the library contains the **Matrix** class and its derivatives: **SubMatrix** and **ConstSubMatrix** classes.

- The **Matrix** class enables construction of matrices with various dimensions and element types (C++ fundamental types or `std::complex`), supporting full matrix arithmetic. It implements all required methods for algorithm functionality along with additional utilities for basic linear algebra operations. Furthermore, through **concepts**, the class facilitates cross-type arithmetic operations, allowing for multiplication/addition/-subtraction between **matrix** and **submatrix** objects.

```
1 LinAlgTools::Matrix<int> matrix = {{1, 2}, {3, 4}};
2 std::cout << matrix; //[1, 2], [3, 4]
3
4 LinAlgTools::Matrix<int> sq_matrix = {2};
5 std::cout << sq_matrix; //[0, 0], [0, 0]
6
7 LinAlgTools::Matrix<int> identity = LinAlgTools::Matrix<int>::Identity(2);
8 std::cout << identity; //[1, 0], [0, 1]
```

- The **SubMatrix** and **ConstSubMatrix** classes allow to perform operations with a particular part of the matrix without copying data. This proves to be particularly useful in our case, enabling application of transformations to a submatrix, greatly reducing the number of unnecessary computations. The classes also allow to create an object of a **matrix** class.

```
1 LinAlgTools::Matrix<int> matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
2 LinAlgTools::SubMatrix<int> submatrix = {matrix, {0, 1}, {0, 1}};
3 std::cout << submatrix; //[1, 2], [4, 5]]1
4
5 submatrix *= 5;
6 std::cout << matrix; //[5, 10, 3], [20, 25, 6], [7, 8, 9]];
```

4.2 Core

The entirety of this part of the library is dedicated towards providing useful functions or other functionality, which is used throughout the project.

- **Math_utils** provide the ability to calculate the sign of a real or a complex number, check whether a matrix is diagonal, orthogonal and much more. **Indices** store uniform types of matrix entry index, row and column slices. This allows to easily change the required type if needed, and the changes will take place throughout the whole library.

```
1 LinAlgTools::Matrix<long double> matrix = {{1e-16, 1e-16}};  
2 LinAlgTools::Matrix<long double> zero_matrix = {{0, 0}};  
3 std::cout << LinAlgTools::AreEqualMatrices(matrix, zero_matrix); //true
```

- **Matrix_traits** and **is_complex** are used to handle different matrix types and complex number respectively. Concepts, mentioned earlier, were introduced in C++ and are a tool, allowing for uniform templates for any matrix class.

```
1 template<Core::MatrixType F, Core::MatrixType S>  
2 Matrix<typename F::ElementType> operator+(const F& lhs, const S& rhs) {  
3     /*Implementation*/  
4 }
```

4.3 Algorithms

This part of the library contains the implementation of the algorithms, which were extensively discussed above. Majority of algorithms' subroutines are implemented as separate functions and can be used as linear transformations. Moreover, the implementation of the algorithms exactly matches its pseudocode counterpart for preserving cohesion and level of abstraction.

- **Householder** contains **HouseholderVectorReduction** for transforming a vector, zeroing out each elements below the first one. **HouseholderLeftReflection** and **HouseholderRightReflection**, provided a vector, are used to apply a reflection to a matrix.
- **Givens** contains **GetGivensPair**, which calculates coefficients for further rotation. **GivensLeftRotation** and **GivensRightRotation**, provided a vector, are used to apply a rotation to a matrix.
- **QR_Decomposition** contains two implementations of the decomposition: one based on Householder reflections and another based on Givens rotations. The former utilizes **HouseholderVectorReduction** and **HouseholderLeftReflection** to transform the matrix to the required form. Reflections are calculated in-place and to a particular submatrix to avoid unnecessary computations and copies. The latter uses **GetGivensPair**, **GivensLeftRotation** and **GivensRightRotation** to calculate matrices Q and R . Left rotations are applied to submatrices of R , while right rotations are applied to submatrices of Q .

A third implementation, **HessenbergQR**, is also provided. Given its conceptual similarity to **GivensQR** and the comparison already presented in the theoretical section, we omit further detailed discussion here.

```
1 LinAlgTools::Matrix<double> matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};  
2 auto [Q_1, R_1] = LinAlgTools::Algorithm::HouseholderQR(matrix);  
3 auto [Q_2, R_2] = LinAlgTools::Algorithm::GivensQR(matrix);  
4 std::cout << LinAlgTools::Core::AreEqualMatrices(Q_1 * R_1, matrix); //true
```

- **HessenbergForm** contains the implementation of the eponymous algorithm. Though similar to **Householder QR Decomposition**, it applies Householder Rotations starting with the second row. Moreover, **HouseholderRightReflections** are also applied to preserve similarity of the matrix.

```
1 LinAlgTools::Matrix<double> matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};  
2 auto [U, H] = LinAlgTools::Algorithm::HessenbergForm(matrix);  
3 auto reconstructed = U * H * U.ConjugateTranspose();  
4 std::cout << LinAlgTools::Core::AreEqualMatrices(reconstructed, matrix); //true
```

5 Testing

The library utilizes a testing framework to validate correctness and measure performance. The test architecture is organized as follows:

```
LinAlgTools/  
├── tests/  
│   ├── benchmark_tests/  
│   │   ├── benchmark_utils/  
│   │   │   ├── algorithm_benchmark.h  
│   │   │   └── random_generator.h  
│   │   └── decomposition_benchmark_tests.cpp  
│   └── validation_tests/  
│       ├── matrix_tests.cpp  
│       ├── sub_matrix_tests.cpp  
│       ├── const_sub_matrix_tests.cpp  
│       ├── qr_decomposition_tests.cpp  
│       ├── schur_decomposition_tests.cpp  
│       └── svd_tests.cpp
```

Figure 3: Directory structure of the **Tests** section

5.1 Validation tests

The majority of implemented functions, classes, and algorithms in the library are evaluated across multiple test cases. Certain functions that form part of a larger algorithm are not individually tested, as the correct operation of the algorithm inherently verifies their functionality. The reliability of the library is further validated using Googletest [4].

All tests related to matrices follow the general structure:

- Test on square matrices.
- Test on rectangular matrices.
- Test on matrices with complex coefficients.
- Test on matrices with ε (small) coefficients.
- Test on potential edge-cases.

In addition to these tests, decomposition algorithms impose further requirements:

- Comparison of the reconstructed matrix to the original, ensuring that the error stays within the range of machine error bounds.
- Verification of decomposition correctness by confirming matrices exhibit the expected structure.

The final validation step is typically implemented as a dedicated function, named `Check{AlgorithmName}`.

5.2 Benchmark tests

All decomposition algorithms are evaluated using randomly generated matrices. For each matrix size within the specified range (from minimum to maximum), every algorithm performs the decomposition on 10 dense matrices. The following performance metrics are recorded for each size in a dedicated CSV file (`algorithm_performance.csv`):

- Matrix size
- Mean execution time
- Minimum execution time

- Maximum execution time
- Standard deviation of timings

The benchmarking parameters are fully configurable:

- Minimum and maximum matrix sizes
- Number of test matrices per size iteration
- Size increment step between tests

Remark. The benchmark tests have been conducted on a MacBook Pro with Apple M2 pro chip and 32gb of RAM.

5.2.1 Householder vs. Givens

The same seed is used in the random number generator to ensure consistent comparisons.

As previously stated, this configuration is ideal for Householder implementation, and the experimental data further supports this claim. Figure 4 demonstrates that as matrix size grows, the mean execution time of the Givens implementation surpasses that of the Householder method, which is an expected outcome given its higher computational complexity. Figure 5 reinforces this observation by displaying the ratio of Givens' mean execution time to Householder's for each matrix size, confirming Givens' inferior performance. However, the performance gap diminishes with increasing matrix size, though the limited sample size prevents definitive conclusions about whether this trend persists for larger matrices.

The theoretical also highlighted Givens' advantage for sparse matrices, but empirical testing failed to validate this claim, so the corresponding data is omitted. The current Givens implementation does not fully leverage sparse matrix structures, necessitating further optimizations, such as those found in *Numpy* [6], to observe meaningful differences. Since such optimizations were beyond the scope of this project, they were not incorporated into LinAlgTools. Nevertheless, the project's scalable design allows for future enhancements to better handle structured matrices.

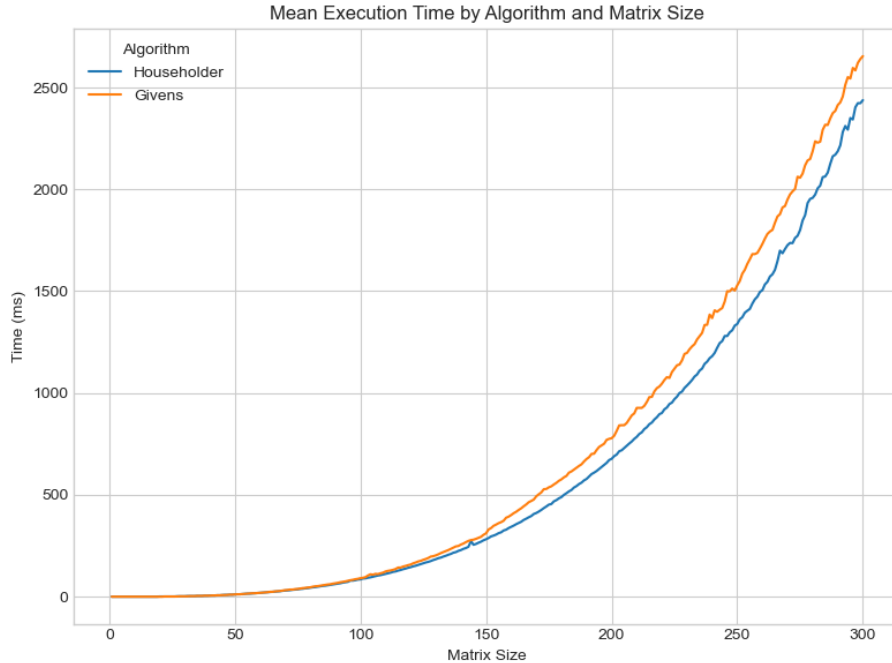


Figure 4: Mean execution time by algorithm and matrix size

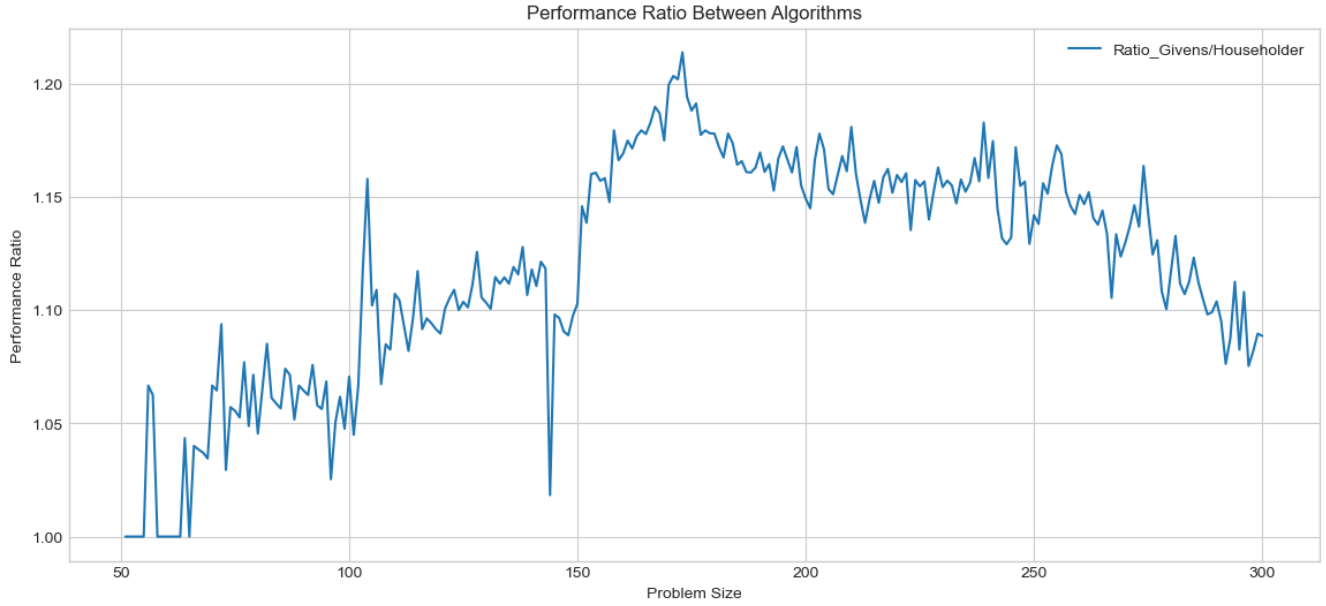


Figure 5: Standard deviation of execution times by algorithm

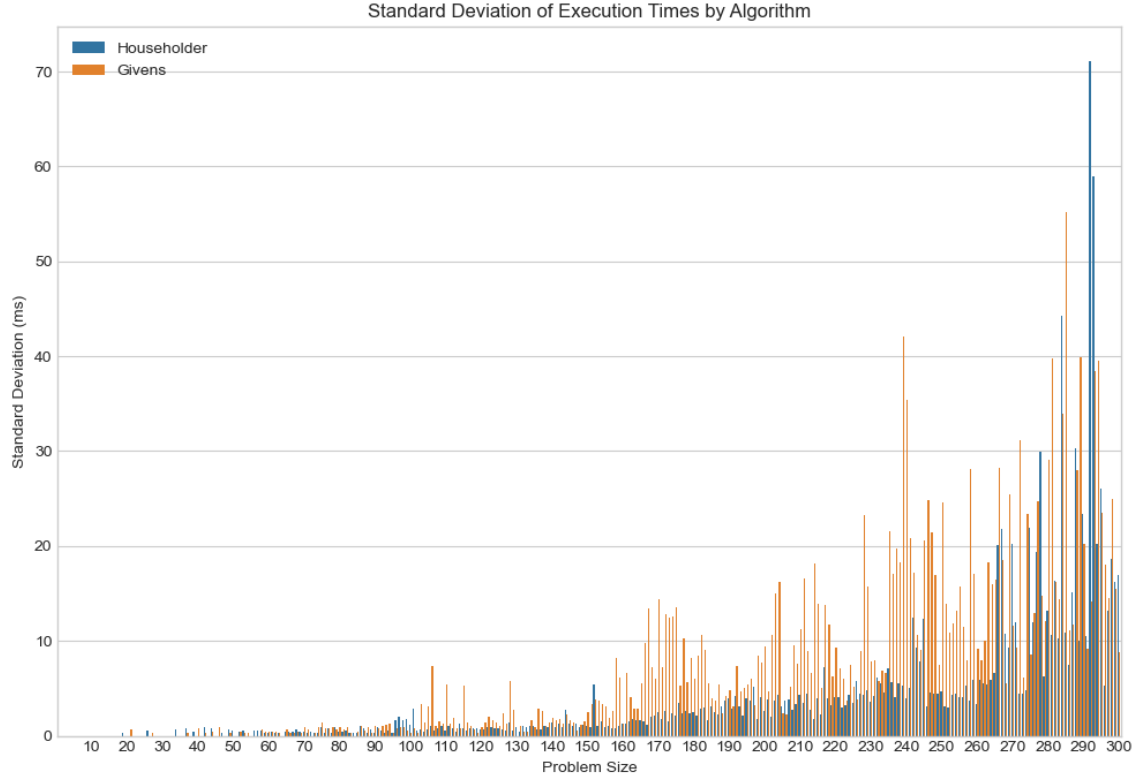


Figure 6: Standard deviation of execution times by algorithm

6 Conclusion

In conclusion, all objectives of the programming project have been successfully achieved. The result of the accomplished work is `LinAlgTools`, a C++ library, offering matrix arithmetic and decomposition algorithms. Rigorous testing and performance benchmarking confirm its reliability, while the intentionally clean, well-structured source, code, publicly available on github, makes it accessible to anyone with some knowledge of C++.

References

- [1] Joseph W. H. Liu Alan George. Householder reflections versus givens rotations in sparse orthogonal decomposition. URL: <https://www.sciencedirect.com/science/article/pii/002437958790111X>, 2002.
- [2] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, third edition, 1999.
- [3] Charles F. Van Loan Gene H. Golub. Matrix computations, 4th edition. URL: <https://doi.org/10.56021/9781421407944>, 2013.
- [4] Google. Googletest. URL: <https://github.com/google/googletest>, 1.15.2.
- [5] Gaël Guennebaud, Benoît Jacob, et al. Eigen v3. URL: <http://eigen.tuxfamily.org>, 2010.
- [6] Millman K.J. van der Walt S.J. et al Harris, C.R. Numpy. URL: <https://github.com/numpy/numpy>, 2025.
- [7] Davis E. King. Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research*, 10:1755–1758, 2009.
- [8] Conrad Sanderson and Ryan Curtin. Armadillo. URL: https://arma.sourceforge.net/armadillo_iccae_2025.pdf, 2025.
- [9] Michelle Schatzman. A simple proof of convergence of the QR algorithm for normal matrices without shifts. <https://conservancy.umn.edu/server/api/core/bitstreams/ec1ba98c-42ec-4f42-8fa9-f3494af09c2d/content>, 1990.
- [10] Tai-Lin Wang and William B. Gragg. Convergence of the shifted QR algorithm for unitary hessenberg matrices. <https://www.ams.org/journals/mcom/2002-71-240/S0025-5718-01-01387-4/S0025-5718-01-01387-4.pdf>, 2001.
- [11] Zinkin Zakhar. LinAlgTools. URL: <https://github.com/Avgustineiw/LinAlgTools>, 2025.